

Planejamento no Mundo dos Blocos de Tamanho Variável: Representação do Conhecimento e Raciocínio Automatizado

Bruna de Souza Brasil, Jamily Oliveira, Roberta Brasil

¹Instituto de Computação (IComp) – Universidade Federal do Amazonas (UFAM)
Av. Gen. Rodrigo Octávio, 6200, Coroado I, 69080-900 – Manaus – AM – Brasil

{bruna.brasil, jamily.oliveira, roberta.brasil}@icomp.ufam.edu.br

Resumo. Este trabalho propõe uma representação em Lógica de Primeira Ordem para o planejamento no Mundo dos Blocos com blocos de tamanhos diferentes, levando em conta posição horizontal, empilhamento e estabilidade. A representação é codificada em CNF e resolvida com um SAT solver, e os planos obtidos são comparados com os planos feitos manualmente em três cenários.

1. Introdução

O Mundo dos Blocos é um domínio clássico de planejamento em Inteligência Artificial. Neste trabalho ele é estendido para blocos de comprimentos diferentes, sobre uma mesa com posições horizontais definidas. Com isso, um bloco pode ocupar vários slots, apoiar-se em mais de um bloco (ponte), deixar espaços vazios embaixo de si e precisa estar equilibrado.

O objetivo é descrever esse mundo em Lógica de Primeira Ordem, construir planos manualmente, traduzir a descrição para CNF e gerar planos com um SAT solver, comparando-os com os planos manuais nas Situações 1, 2 e 3.

2. Descrição Formal do Mundo dos Blocos de Tamanho Variado

2.1. Domínio

Blocos	$B = \{a, b, c, d\}$, mesa T
Comprimentos	$\ell(a) = \ell(b) = 1$, $\ell(c) = 2$, $\ell(d) = 3$
Pontos	$X = \{0, 1, \dots, 6\}$
Slots	$s_i = [i, i + 1]$, $i \in \{0, \dots, 5\}$
Níveis	$l \in \{0, 1, 2, 3\}$ (0 = mesa)
Tempo	$t \in \{0, \dots, H\}$, onde H é o horizonte (número de passos)
Span	$span(b, p) = \{p, \dots, p + \ell(b) - 1\}$, com $0 \leq p \leq 6 - \ell(b)$

2.2. Predicados

Predicado	Significado
$at(b, p, t)$	b começa no ponto p
$lev(b, l, t)$	b está no nível l
$cov(b, s, l, t)$	b ocupa o slot s no nível l
$overlap(b, p, y, q)$	spans de b (em p) e y (em q) se sobrepõem
$on(b, y, t)$	b está apoiado em $y \in B \cup \{T\}$
$clr(b, t)$	nada está sobre b

Apenas *at* e *lev* são fluentes básicos. Os demais são derivados:

$$\begin{aligned}
cov(b, s, l, t) &\leftrightarrow \exists p (at(b, p, t) \wedge lev(b, l, t) \wedge s \in span(b, p)) \\
overlap(b, p, y, q) &\leftrightarrow p < q + \ell(y) \wedge q < p + \ell(b) \\
on(b, T, t) &\leftrightarrow lev(b, 0, t) \\
on(b, y, t) &\leftrightarrow \exists p, q, l (at(b, p, t) \wedge at(y, q, t) \wedge lev(b, l, t) \\
&\quad \wedge lev(y, l - 1, t) \wedge overlap(b, p, y, q)) \\
clr(b, t) &\leftrightarrow \neg \exists x on(x, b, t)
\end{aligned}$$

2.3. Restrições de estado

$$\text{Unicidade: } \forall b \exists! p at(b, p, t) \quad \forall b \exists! l lev(b, l, t)$$

$$\text{Exclusão: } \forall b \neq b', s, l \neg (cov(b, s, l, t) \wedge cov(b', s, l, t))$$

$$\begin{aligned}
\text{Estabilidade: } at(b, p, t) \wedge lev(b, l, t) \wedge l > 0 \rightarrow \\
|\{s \in span(b, p) : \exists b' \neq b cov(b', s, l - 1, t)\}| \geq \lceil \ell(b)/2 \rceil
\end{aligned}$$

A contagem equivale a uma disjunção finita. Assim, *d* precisa de 2 slots apoiados (o que permite a ponte) e *c* precisa de 1.

2.4. A ação move

move(b, y, p, t): mover *b* para cima de *y* (bloco ou mesa), começando no ponto *p*. O nível de destino é $l^* = 0$ se $y = T$, e $l^* = lev(y) + 1$ se $y \in B$. Na codificação CNF (Seção 3), cada ação corresponde à variável *mv(b, y, p, t)*.

	Precondição	Manual	Fórmula
P1	<i>b</i> livre	1	<i>clr(b, t)</i>
P2	destino válido	–	$y \neq b, 0 \leq p \leq 6 - \ell(b)$
P3	sobreposição com <i>y</i>	4	$y \in B \rightarrow at(y, q, t) \wedge overlap(b, p, y, q)$
P4	slots livres	5	$\forall s \in span(b, p), z \neq b \neg cov(z, s, l^*, t)$
P5	folga vertical	nova	$\forall s \in span(b, p), l > l^*, z \neq b \neg cov(z, s, l, t)$
P6	estabilidade	–	regra de estabilidade em (p, l^*) , se $l^* > 0$
P7	ação não nula	3	$\neg (at(b, p, t) \wedge lev(b, l^*, t))$

A coluna “Manual” indica a precondição correspondente no Manual. A precondição 2 do Manual (*clr(y)*) foi removida (Seção 2.7).

Efeitos	(posição anterior de <i>b</i> : p_0, l_0)
<i>add</i>	$at(b, p, t + 1), lev(b, l^*, t + 1)$
<i>add</i>	<i>clr(z, t + 1)</i> para cada antigo apoio <i>z</i> que ficou livre
<i>delete</i>	$at(b, p_0, t + 1)$ se $p_0 \neq p$; $lev(b, l_0, t + 1)$ se $l_0 \neq l^*$
<i>delete</i>	<i>clr(y', t + 1)</i> para todo <i>y'</i> sob o novo span (todos os apoios)

Persistência e ação única. O que não é afetado pela ação continua igual, e ocorre exatamente uma ação por instante:

$$\begin{aligned}
at(b, p, t) \wedge \neg at(b, p, t + 1) &\rightarrow \bigvee_{y, p'} move(b, y, p', t) \quad (\text{análogo para } lev) \\
\bigvee_{\alpha} move(\alpha, t) &\quad \neg move(\alpha, t) \vee \neg move(\beta, t) \quad \text{para ações distintas } \alpha, \beta
\end{aligned}$$

Com isso, o plano tem exatamente H ações, e o menor H satisfatível é o tamanho do plano mínimo.

2.5. Elementos novos e ações associadas

Elemento	Tipo	Ações associadas
$\ell(b)$	estático	nenhuma
$at(b, p, t)$	fluente	<i>add</i> por $move(b, \cdot, p, t)$; <i>delete</i> da posição antiga
$lev(b, l, t)$	fluente	<i>add</i> de l^* por $move(b, y, \cdot, t)$; <i>delete</i> do nível antigo
$clr(y, t)$	fluente	<i>delete</i> quando um bloco passa a cobrir y (inclusive ponte); <i>add</i> quando o último bloco sobre y sai
$cov, overlap$	derivados	mudam junto com at e lev
$on(b, y, t)$	derivado	muda só por $move(b, \cdot, \cdot, t)$
estabilidade	invariante	checada em P6; preservada, pois apoio nunca está livre

2.6. Ordem parcial

$\varphi_1 \prec_P \varphi_2$ exige que φ_1 ocorra antes de φ_2 :

$$\forall t (\varphi_2(t) \rightarrow \exists t' < t \varphi_1(t'))$$

Exemplo usado: $\varphi_1(t) \equiv lev(d, 0, t)$ (d na mesa) e $\varphi_2(t) \equiv on(a, c, t)$ (a sobre c). Em CNF, com o_t auxiliar para $on(a, c, t)$:

$$\neg o_t \vee lev(d, 0, 0) \vee \dots \vee lev(d, 0, t-1) \\ \neg at(a, p, t) \vee \neg lev(a, l, t) \vee \neg at(c, q, t) \vee \neg lev(c, l-1, t) \vee o_t$$

a segunda cláusula para cada p, q com spans sobrepostos. A precedência é estrita ($t' < t$). Em $t = 0$ a primeira cláusula fica só $\neg o_0$.

2.7. Ajustes em relação ao Manual

Ajuste	Motivo
Remoção de $clr(y)$	Impede dois blocos lado a lado sobre um bloco maior e torna S_{f1} e S_{f2} (Situação 1) e S_5 (Situação 2) inalcançáveis. P4 já garante os slots livres. Cláusulas na Seção 3.3.
Folga vertical (P5)	O Manual proíbe ocupar o slot sob a ponte, mas não formaliza. Sem P5, S_{f3} sairia em 2 ações deslizando a sob d . Cláusulas na Seção 3.4.
Efeitos completos	Liberar o antigo apoio e ocupar todos os apoios da ponte.

3. Codificação CNF

Para usar um SAT solver, o mundo descrito em lógica de primeira ordem é traduzido para lógica proposicional. Cada fato possível do mundo vira uma variável que só pode ser verdadeira ou falsa, e as regras do mundo viram cláusulas sobre essas variáveis. Se existe uma atribuição que satisfaz todas as cláusulas, ela contém o plano. Neste texto, T é o número de passos do plano e também o símbolo da mesa, como no Manual; o contexto indica qual dos dois é.

3.1. Variáveis proposicionais

Para um horizonte de T passos, são criadas quatro famílias de variáveis (Tabela 1).

Variável	Significado	Índices
$at(b, p, t)$	o bloco b começa no ponto p no instante t	$b \in B, p \in [0, 6 - \ell(b)], t \in [0, T]$
$lev(b, l, t)$	b está no nível l no instante t	$l \in [0, 3]$
$clr(b, t)$	o topo de b está livre no instante t	$t \in [0, T]$
$mv(b, y, p, t)$	em t , b é movido para cima de y (ou da mesa T), começando em p ; o efeito vale em $t + 1$	$y \in (B \cup \{T\}) \setminus \{b\}, t \in [0, T - 1]$

Tabela 1. Variáveis proposicionais da codificação.

A relação *on* não é codificada. Ela é derivada depois, a partir de *at*, *lev* e da sobreposição dos spans (Seção 7).

Na Situação 2, com $T = 5$, os blocos a, b, c e d têm 6, 6, 5 e 4 posições válidas, somando 21. Então *at* tem $21 \cdot 6 = 126$ variáveis, *lev* tem $4 \cdot 4 \cdot 6 = 96$, *clr* tem $4 \cdot 6 = 24$ e *mv* tem $4 \cdot 21 \cdot 5 = 420$, num total de 666 variáveis. As variáveis auxiliares *cov* do grupo 6 ficam fora dessa conta.

3.2. Grupos de cláusulas

1. **Estado inicial.** Cláusulas unitárias que fixam *at* e *lev* em $t = 0$ com as posições do estado inicial. Exemplo: $at(c, 0, 0)$ e $lev(c, 0, 0)$.
2. **Meta.** Cláusulas unitárias que fixam *at* e *lev* em $t = T$ com as posições do estado final.
3. **Unicidade de posição.** Em cada instante, cada bloco tem exatamente um ponto inicial. Pelo menos um: $\bigvee_p at(b, p, t)$. No máximo um: $\neg at(b, p_1, t) \vee \neg at(b, p_2, t)$ para $p_1 < p_2$.
4. **Unicidade de nível.** Mesma ideia para *lev*: cada bloco está em exatamente um nível em cada instante.
5. **Exclusão horizontal.** Dois blocos no mesmo nível não podem cobrir o mesmo slot. Para $b \neq b'$ com spans que se sobrepõem em p e p' :

$$\neg at(b, p, t) \vee \neg lev(b, l, t) \vee \neg at(b', p', t) \vee \neg lev(b', l, t).$$

6. **Estabilidade.** Um bloco no nível $l > 0$ precisa ter pelo menos $\lceil \ell(b)/2 \rceil$ dos slots do seu span ocupados no nível $l - 1$. Seja $cov(s, l, t)$ a afirmação “o slot s é coberto por algum bloco no nível l no instante t ”. Para cada $b, p, l > 0$ e cada conjunto S de slots do span com $|S| = \ell(b) - \lceil \ell(b)/2 \rceil + 1$:

$$\neg at(b, p, t) \vee \neg lev(b, l, t) \vee \bigvee_{s \in S} cov(s, l - 1, t).$$

Para a e b ($\ell = 1$) sai uma cláusula com um único *cov*. Para c ($\ell = 2$), uma cláusula com os dois slots. Para d ($\ell = 3$), uma cláusula por par de slots, o que exige pelo menos dois dos três apoios. Como *cov* é uma disjunção de conjunções $at \wedge lev$, no CNF ele vira uma variável auxiliar definida por equivalência. Blocos no nível 0 não precisam de apoio, pois estão na mesa.

7. **Clear.** $clr(b, t)$ é verdadeira exatamente quando nenhum bloco está no nível acima de b cobrindo algum slot do seu span. Para b em (p, l) :

$$\neg at(b, p, t) \vee \neg lev(b, l, t) \vee \neg cov(s, l + 1, t) \vee \neg clr(b, t) \quad \text{para cada } s \in \text{span}(b, p),$$

$$clr(b, t) \vee \neg at(b, p, t) \vee \neg lev(b, l, t) \vee \bigvee_{s \in \text{span}(b, p)} cov(s, l + 1, t).$$

Como clr é definida em todo instante por essas cláusulas, ela se atualiza sozinha quando um bloco sai de cima de outro, inclusive nos dois apoios de uma ponte.

8. **Pré-condições de $mv(b, y, p, t)$.** Sendo m o nível de y (o nível-alvo é $m + 1$, ou 0 se $y = T$):

- (a) b tem o topo livre: $\neg mv(b, y, p, t) \vee clr(b, t)$.
- (b) (Ajustada, ver Seção 3.3.)
- (c) O destino não é a posição atual: $\neg mv(b, y, p, t) \vee \neg at(b, p, t) \vee \neg lev(y, m, t) \vee \neg lev(b, m + 1, t)$. Para $y = T$, a cláusula é $\neg mv(b, T, p, t) \vee \neg at(b, p, t) \vee \neg lev(b, 0, t)$.
- (d) Se y é bloco, o span de b em p sobrepõe o de y : $\neg mv(b, y, p, t) \vee \neg at(y, q, t)$ para cada q cujo span não sobrepõe o de b em p .
- (e) Os slots cobertos por b no nível-alvo estão livres. Para cada $x \neq b$ em $(q, m + 1)$ com span sobrepondo o de b em p : $\neg mv(b, y, p, t) \vee \neg lev(y, m, t) \vee \neg at(x, q, t) \vee \neg lev(x, m + 1, t)$. Para $y = T$: $\neg mv(b, T, p, t) \vee \neg at(x, q, t) \vee \neg lev(x, 0, t)$.

A estabilidade do destino não precisa de cláusula própria aqui, pois o grupo 6 aplicado em $t + 1$ já a exige. A cláusula do slot sob a ponte, que o Manual não traz, está na Seção 3.4.

9. **Efeitos de $mv(b, y, p, t)$.**

$$mv(b, y, p, t) \rightarrow at(b, p, t + 1),$$

$$mv(b, y, p, t) \wedge lev(y, m, t) \rightarrow lev(b, m + 1, t + 1) \quad (\text{ou } lev(b, 0, t + 1) \text{ se } y = T),$$

$$mv(b, y, p, t) \rightarrow \neg clr(y, t + 1) \quad (\text{se } y \text{ é bloco}).$$

Que b deixa a posição e o nível anteriores segue da unicidade (grupos 3 e 4). Que os apoios antigos de b voltam a ficar livres segue do grupo 7.

10. **Frame axioms.** O que não muda no passo continua igual. Seja $M_b(t) = \bigvee_{y, p} mv(b, y, p, t)$. Então

$$at(b, p, t) \wedge \neg M_b(t) \rightarrow at(b, p, t + 1) \quad \text{e} \quad lev(b, l, t) \wedge \neg M_b(t) \rightarrow lev(b, l, t + 1).$$

Para clr não é preciso axioma próprio, pois o grupo 7 a define em todo instante.

11. **Ação única por passo.** Em cada t exatamente uma variável mv é verdadeira: pelo menos uma, $\bigvee mv(\cdot, \cdot, \cdot, t)$, e no máximo uma, $\neg mv_i \vee \neg mv_j$ para cada par de variáveis mv do instante t .

3.3. Ajuste na pré-condição 2 de move

A pré-condição 2 do Manual exige $clr(y, t)$, isto é, que o topo inteiro de y esteja livre. Com essa leitura literal, nenhum bloco poderia ser colocado sobre um y que já carrega outro bloco, mesmo em uma parte livre do seu topo.

Isso impede a Situação 2. Em S_0 e em S_5 , os blocos a e b estão lado a lado sobre c . Em S_5 , o primeiro dos dois a ser colocado sobre c deixaria $clr(c)$ falso, e a pré-condição

literal bloquearia o segundo. Como c é o único bloco do nível 1 que serve de apoio aos dois, S_5 não seria alcançável a partir de S_0 .

O ajuste adotado é retirar $clr(y, t)$ como condição independente. O que realmente importa é que os slots onde b vai pousar estejam livres, e isso já é exigido pela pré-condição 5. O bloco que se move continua precisando de $clr(b, t)$, e o efeito $\neg clr(y, t + 1)$ continua correto, pois y passa a ter b em cima.

3.4. Cláusula do slot sob a ponte

No estado inicial da Situação 1, o bloco d está em ponte sobre a e b , com o slot s_4 vazio no nível 0 e coberto por d no nível 1. A pré-condição 5 só olha o nível-alvo. Se o nível-alvo é o 0, o slot s_4 aparece livre, e nada impediria colocar um bloco ali embaixo de d . O modelo do Manual considera esse slot bloqueado, mas nenhuma cláusula diz isso.

A regra geral é: um bloco não pode pousar em um slot do nível l se algum bloco em um nível maior que l cobre esse slot. Se o slot já tem bloco no nível l , a pré-condição 5 já impede. A cláusula nova cobre o caso do slot vazio. Para cada $x \neq b$ em (q, k) com span sobrepondo o de b em p e $k > m + 1$:

$$\neg mv(b, y, p, t) \vee \neg lev(y, m, t) \vee \neg at(x, q, t) \vee \neg lev(x, k, t).$$

Para $y = T$, o nível-alvo é 0 e vale, para todo $k \geq 1$:

$$\neg mv(b, T, p, t) \vee \neg at(x, q, t) \vee \neg lev(x, k, t).$$

4. Exemplos dos Três Cenários

4.1. Situação 1

4.1.1. Estados inicial e meta

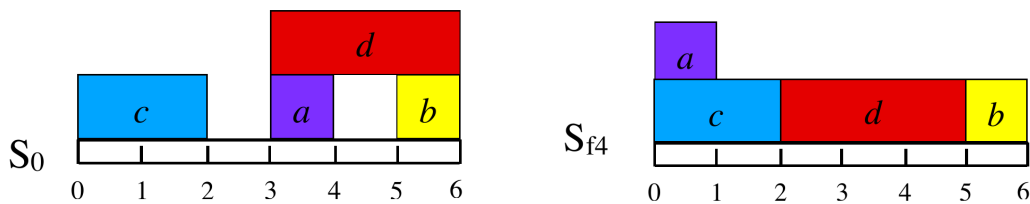


Figura 1. Estado inicial S_0 e meta S_{f4} da Situação 1.

$$\begin{aligned} S_0 : & at(c, 0) \wedge lev(c, 0) \wedge at(a, 3) \wedge lev(a, 0) \\ & \wedge at(b, 5) \wedge lev(b, 0) \wedge at(d, 3) \wedge lev(d, 1) \\ S_{f4} : & at(c, 0) \wedge lev(c, 0) \wedge at(a, 0) \wedge lev(a, 1) \\ & \wedge at(d, 2) \wedge lev(d, 0) \wedge at(b, 5) \wedge lev(b, 0) \end{aligned}$$

Estado	Relações <i>on</i>
S_0	$on(c, T), on(a, T), on(b, T), on(d, a), on(d, b)$
S_{f4}	$on(c, T), on(d, T), on(b, T), on(a, c)$

4.1.2. Plano manual

Passo	Ação	Verificação
$t = 0$	$move(d, c, 0)$	d livre; nível 1, slots 0–2 livres; apoio em 0 e 1 ($2 \geq 2$).
$t = 1$	$move(a, b, 5)$	a livre; slot 5 livre no nível 1; apoio em b .
$t = 2$	$move(d, T, 2)$	d livre; slots 2–4 livres no nível 0, nada acima.
$t = 3$	$move(a, c, 0)$	a livre; slot 0 livre no nível 1; apoio em c . Em $t = 4$ o estado é S_{f4} .

Tabela 2. Plano manual da Situação 1, de S_0 até S_{f4} .

Por que 4 ações é o mínimo. d precisa de 2 ações (não há 3 slots livres consecutivos em S_0). a também precisa de 2, pois só fica livre quando d vai para $p = 0$ sobre c , cobrindo o destino de a . Logo, 4 ações é o mínimo, confirmado por busca exaustiva.

Ordem parcial. d chega à mesa em $t = 3$ e a fica sobre c em $t = 4$, então $\varphi_1 \prec_P \varphi_2$ (Seção 2.6) é respeitada.

Outras metas. Planos mínimos por busca exaustiva: S_{f1} com 9 ações, S_{f2} com 10 e S_{f3} com 10.

4.1.3. Codificação em CNF

Os grupos 1 e 2 são as oito cláusulas unitárias de S_0 (em $t = 0$) e as oito de S_{f4} (em $t = 4$), escritas acima. Com $H = 4$ há $21 \cdot 5 = 105$ variáveis at , $4 \cdot 4 \cdot 5 = 80$ de lev , $4 \cdot 5 = 20$ de clr e $84 \cdot 4 = 336$ de mv , num total de 541, mais as auxiliares cov e as $o_0, \dots, o_4$ da ordem parcial. No script:

```
INITIAL = { 'c' : (0, 0), 'a' : (3, 0), 'b' : (5, 0), 'd' : (3, 1) }
GOAL    = { 'c' : (0, 0), 'a' : (0, 1), 'd' : (2, 0), 'b' : (5, 0) }
HORIZON = 4
```

Pelo plano manual, o resultado esperado é UNSATISFIABLE para $H = 0, \dots, 3$ e SATISFIABLE para $H = 4$, com ou sem a cláusula de ordem parcial.

4.2. Situação 2

4.2.1. Estados inicial e meta

Cada ponto é dado como (p, l) , com p o ponto inicial e l o nível. O estado inicial S_0 é

$$\begin{aligned} at(c, 0, 0) \wedge lev(c, 0, 0), & \quad at(d, 3, 0) \wedge lev(d, 0, 0), \\ at(a, 0, 0) \wedge lev(a, 1, 0), & \quad at(b, 1, 0) \wedge lev(b, 1, 0), \end{aligned}$$

com as relações $on(c, T, 0)$, $on(d, T, 0)$, $on(a, c, 0)$ e $on(b, c, 0)$. O slot s_2 da mesa está vazio e livre.

O estado meta S_5 , em $t = 5$, é

$$\begin{aligned} at(d, 3, 5) \wedge lev(d, 0, 5), & \quad at(c, 4, 5) \wedge lev(c, 1, 5), \\ at(a, 4, 5) \wedge lev(a, 2, 5), & \quad at(b, 5, 5) \wedge lev(b, 2, 5), \end{aligned}$$

com as relações $on(d, T, 5)$, $on(c, d, 5)$, $on(a, c, 5)$ e $on(b, c, 5)$.

4.2.2. Plano manual

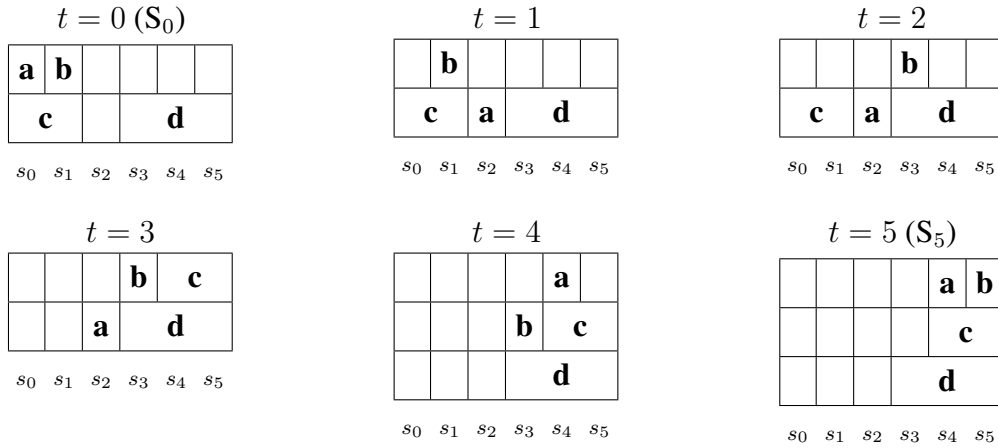


Figura 2. Estados da Situação 2 ao longo do plano manual. As colunas são os slots s_0 a s_5 e as linhas são os níveis (a mesa é o nível 0).

Passo	Ação	Verificação
$t = 0$	$mv(a, T, 2, 0)$	a tem o topo livre. O destino é a mesa e o slot s_2 está vazio, sem bloco acima dele. Em $t = 1$, a está fora de c e b continua sobre c .
$t = 1$	$mv(b, d, 3, 1)$	b tem o topo livre. Em $p = 3$ ele sobrepõe d e o slot s_3 do nível 1 está livre. Apoio: s_3 está sobre d , e $1 \geq \lceil 1/2 \rceil$. Agora c fica com o topo livre.
$t = 2$	$mv(c, d, 4, 2)$	c tem o topo livre, pois a e b já saíram. Em $p = 4$ ele cobre s_4 e s_5 , livres no nível 1 (b está em s_3). Apoio: os dois slots estão sobre d , e $2 \geq \lceil 2/2 \rceil$. Aqui d não tem o topo totalmente livre, o que a pré-condição 2 original proibiria.
$t = 3$	$mv(a, c, 4, 3)$	a tem o topo livre. Em $p = 4$ ele sobrepõe c e o slot s_4 do nível 2 está livre. Apoio: s_4 está sobre c , e $1 \geq \lceil 1/2 \rceil$.
$t = 4$	$mv(b, c, 5, 4)$	b tem o topo livre. Em $p = 5$ ele sobrepõe c e o slot s_5 do nível 2 está livre. Apoio: s_5 está sobre c , e $1 \geq \lceil 1/2 \rceil$. Em $t = 5$ o estado é S_5 .

Tabela 3. Plano manual da Situação 2, de S_0 até S_5 .

Por que 5 ações é o mínimo. Como a e b começam sobre c e c só se move com o topo livre, os dois precisam sair antes de c se mover. Como terminam sobre c em $(4, 1)$, cada

um precisa de um segundo movimento depois que c chegou lá. Logo a e b se movem pelo menos duas vezes cada, e c pelo menos uma, o que dá $2 + 2 + 1 = 5$ ações. O bloco d não precisa se mover.

Ordem parcial. As metas $\varphi_1 = on(c, d)$, $\varphi_2 = on(a, c)$ e $\varphi_3 = on(b, c)$ obedecem a $\varphi_1 \prec_P \varphi_2$ e $\varphi_1 \prec_P \varphi_3$, pois a e b só ficam sobre c depois que c está em $(4, 1)$. No plano, φ_1 passa a valer em $t = 3$, φ_2 em $t = 4$ e φ_3 em $t = 5$. Entre φ_2 e φ_3 não há ordem: b antes de a também leva a S_5 com 5 ações.

4.2.3. Codificação em CNF

As cláusulas do grupo 1 são as oito cláusulas unitárias de S_0 e as do grupo 2 são as oito de S_5 com $T = 5$, escritas acima. No script, o cenário é configurado assim:

```
INITIAL = { 'a' : (0, 1), 'b' : (1, 1), 'c' : (0, 0), 'd' : (3, 0) }
GOAL    = { 'a' : (4, 2), 'b' : (5, 2), 'c' : (4, 1), 'd' : (3, 0) }
HORIZON = 5
```

Pelo plano manual, o resultado esperado é UNSATISFIABLE para $T = 0, \dots, 4$ e SATISFIABLE para $T = 5$. Esse resultado deve ser confirmado na execução descrita na Seção 6.

4.3. Situação 3

4.3.1. Estados inicial e meta

A Situação 3 parte do mesmo estado inicial S_0 da Situação 1 (bloco d em ponte sobre a e b) e tem como meta o estado S_7 : a e b lado a lado sobre c , e d na mesa, em $p = 3$. A Figura 3 mostra os dois estados.

$$\begin{aligned}
S_0 : & at(c, 0) \wedge lev(c, 0) \wedge at(a, 3) \wedge lev(a, 0) \\
& \wedge at(b, 5) \wedge lev(b, 0) \wedge at(d, 3) \wedge lev(d, 1) \\
S_7 : & at(c, 0) \wedge lev(c, 0) \wedge at(a, 0) \wedge lev(a, 1) \\
& \wedge at(b, 1) \wedge lev(b, 1) \wedge at(d, 3) \wedge lev(d, 0)
\end{aligned}$$

Estado	Relações on
S_0	$on(c, T), on(a, T), on(b, T), on(d, a), on(d, b)$
S_7	$on(c, T), on(d, T), on(a, c), on(b, c)$

Conferência de consistência (restrições da Seção 2.3): em S_0 e S_7 todo bloco respeita $0 \leq p \leq 6 - \ell(b)$; não há dois blocos cobrindo o mesmo slot no mesmo nível; d em S_0 tem apoio em s_3 e s_5 ($2 \geq \lceil 3/2 \rceil$), com s_4 vazio sob a ponte; em S_7 , a e b estão apoiados em c ($1 \geq \lceil 1/2 \rceil$ cada).



Figura 3. Estado inicial S_0 e meta S_7 da Situação 3.

Passo	Ação	Verificação
$t = 0$	$move(d, c, 0)$	d tem o topo livre. Destino: nível 1 sobre c , slots s_0-s_2 livres (e nada acima). Apoio em s_0 e s_1 ($2 \geq 2$), sem precisar de s_2 : é a ponte parcial. Libera a e b .
$t = 1$	$move(a, b, 5)$	a livre. Sobrepõe b em s_5 ; s_5 no nível 1 livre. Apoio em b . Tira a do slot s_3 da mesa.
$t = 2$	$move(d, T, 2)$	d livre. Slots s_2, s_3, s_4 livres no nível 0 e nada acima deles (a está em s_5). Libera o topo de c .
$t = 3$	$move(a, c, 0)$	a livre. Sobrepõe c em s_0 ; s_0 no nível 1 livre. Apoio em c .
$t = 4$	$move(b, c, 1)$	b livre (pois a saiu). Sobrepõe c em s_1 ; s_1 no nível 1 livre. Apoio em c . Aqui c já carrega a e não tem o topo inteiro livre, o que a pré-condição 2 original proibiria (Seção 3.3).
$t = 5$	$move(d, T, 3)$	d livre. Slots s_3, s_4, s_5 livres no nível 0 (a e b já saíram da mesa). Em $t = 6$ o estado é S_7 .

Tabela 4. Plano manual da Situação 3, de S_0 até S_7 .

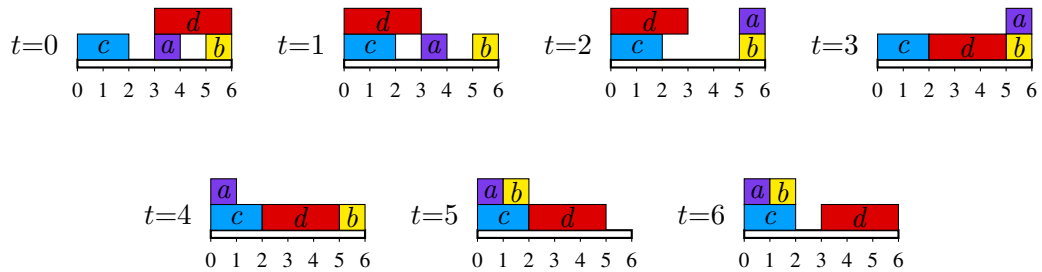


Figura 4. Estados da Situação 3 ao longo do plano manual. Correspondem às figuras $S_0, S_1, S_2, S_3, S_4, S_5$ e S_7 do enunciado.

4.3.2. Plano manual

Por que 6 ações é o mínimo. d não pode ir direto de $(3, 1)$ para $(3, 0)$, pois a e b ocupam os slots s_3 e s_5 do nível 0 e só ficam livres depois que d sai de cima deles. Como o topo de c precisa estar livre quando a e b forem para lá, d não pode ficar sobre c ; e para sair de cima de c para a mesa em $p = 2$, a precisa antes deixar s_3 . Isso dá três ações para d , duas para a e uma para b . A busca exaustiva (BFS) sobre as mesmas regras do modelo confirma que não existe plano com menos de 6 ações.

Ordem parcial. Com $\varphi_1(t) \equiv lev(d, 0, t)$ e $\varphi_2(t) \equiv on(a, c, t)$ (Seção 2.6), φ_1 vale pela primeira vez em $t = 3$ e φ_2 em $t = 4$, logo $\varphi_1 \prec_P \varphi_2$ é respeitada com precedência estrita.

Observação sobre o enunciado. Na figura da Situação 3, S_5 e S_6 são desenhados de forma idêntica. O plano acima passa por $S_0, S_1, S_2, S_3, S_4, S_5$ e vai direto a S_7 , sem um estado repetido.

4.3.3. Codificação em CNF

Os grupos 1 e 2 são as oito cláusulas unitárias de S_0 (em $t = 0$) e as oito de S_7 (em $t = H$), escritas acima. Com $H = 6$ há $21 \cdot 7 = 147$ variáveis at , $4 \cdot 4 \cdot 7 = 112$ de lev , $4 \cdot 7 = 28$ de clr e $84 \cdot 6 = 504$ de mv , num total de 791, mais as auxiliares cov e $o_0, \dots, o_6$ da ordem parcial. No script:

```
INITIAL = {'c' : (0, 0), 'a' : (3, 0), 'b' : (5, 0), 'd' : (3, 1)}
GOAL    = {'c' : (0, 0), 'a' : (0, 1), 'b' : (1, 1), 'd' : (3, 0)}
HORIZON = 6
```

Pelo plano manual, o resultado esperado é UNSATISFIABLE para $H = 0, \dots, 5$ e SATISFIABLE para $H = 6$, com ou sem a cláusula de ordem parcial. Esse resultado deve ser confirmado na execução descrita na Seção 6.

5. Mapeamento: Descrição Formal para Código

Esta seção liga cada elemento da descrição formal (Seções 2 e 3) ao trecho correspondente do código. O gerador de CNF é o `bw2cnf_var.py`, adaptado do caso simples; a leitura do resultado é feita pelo `interpretar.py`. Os nomes em fonte monoespaçada são funções, dicionários ou variáveis do código.

5.1. Domínio, estados e cenários

Cada cenário é descrito por `initial`, `goal`, `ordem` e `esperado` (comprimento mínimo do plano, vindo do plano manual). Por exemplo, a Situação 3 é:

```
INITIAL = {'c' : (0, 0), 'a' : (3, 0), 'b' : (5, 0), 'd' : (3, 1)}
GOAL    = {'c' : (0, 0), 'a' : (0, 1), 'b' : (1, 1), 'd' : (3, 0)}
HORIZON = 6
ordem    = [ (('lev', 'd', 0), ('on', 'a', 'c')) ] # d na mesa < a sobre c
```

5.2. Variáveis proposicionais

Cada variável recebe um número inteiro por `new_var()`, que também guarda o nome simbólico. Esse registro é o que vira o arquivo `.map`.

5.3. Grupos de cláusulas

5.4. O que muda em relação ao caso simples

- **Ação qualitativa.** A variável `mv` tem o destino simbólico y (bloco ou mesa) e o ponto p , em vez de nível e posição numéricos.
- **Sem a variável `on`.** A relação é derivada depois, o que reduz o número de variáveis e permite listar a ponte como dois apoios.
- **Variáveis auxiliares.** `cb` e `cov` permitem escrever estabilidade e clear como cláusulas curtas; `pos` e `o_on` servem à ordem parcial.
- **Ajustes do Manual** (Seção 2.7): sem `clr(y)` como pré-condição; cláusula do slot sob a ponte; efeitos completos (apoio antigo e todos os apoios).
- **Nível máximo.** Se y está no nível 3, $mv(b, y, p, t)$ é proibido, pois o destino seria o nível 4.
- **Cenário e saída.** O cenário é escolhido por nome, e os arquivos vão automaticamente para a pasta do cenário.

5.5. Tamanho do CNF

6. Execução Passo a Passo

O fluxo tem quatro passos. Os exemplos usam a Situação 3; para as outras, troque o cenário.

1. Configurar o cenário. No topo do `bw2cnf_var.py`, escolha o cenário em `CENARIO_PADRAO('sit1_sf4', 'sit2' ou 'sit3')` ou passe `--cenario` na linha de comando. Para a Situação 3, o script usa:

```
INITIAL = {'c': (0,0), 'a': (3,0), 'b': (5,0), 'd': (3,1)}
GOAL     = {'c': (0,0), 'a': (0,1), 'b': (1,1), 'd': (3,0)}
HORIZON  = 6
```

2. Gerar o CNF e o mapa.

```
python3 bw2cnf_var.py --cenario sit3
```

Saída:

Pasta: `situacao3/`

Gerado: 1638 variaveis, 46534 clausulas (base: 791; auxiliares: 847), horizonte T=6
Arquivos: `situacao3/trab01_blocos2SAT.cnf`, `situacao3/trab01_blocos2SAT.map`

O `.cnf` está no formato DIMACS (primeira linha `p cnf 1638 46534`, uma cláusula por linha terminada em 0). O `.map` tem uma linha `ID simbolo por variável`, por exemplo `4 at(a,3,0)`. O argumento `--horizon` muda H (`--cenario sit3 --horizon 5`); `--sem-ordem` desliga a cláusula de ordem parcial.

Descrição formal	Código (bw2cnf_var.py)
Blocos e comprimentos $\ell(b)$	BLOCKS = {'a':1, 'b':1, 'c':2, 'd':3}
Pontos $X = \{0, \dots, 6\}$ e 6 slots	MAX_POINT = 6
Níveis $l \in \{0, 1, 2, 3\}$	MAX_LEVEL = 3
Mesa T	TABLE = 'T' (evita confundir com o horizonte T)
Posições válidas $0 \leq p \leq 6 - \ell(b)$	valid_positions(b)
$span(b, p) = \{p, \dots, p + \ell(b) - 1\}$	span(b, p)
$overlap(b, p, y, q)$	spans_overlap(b1, p1, b2, p2)
Estado inicial e meta	dicionários INITIAL e GOAL: bloco $\mapsto$ (ponto, nível)
Horizonte H (número de ações)	HORIZON; parâmetro T de build()
Situações 1, 2 e 3	dicionário CENARIOS (sit1_sf1 a sit1_sf4, sit2, sit3); escolha em CENARIO_PADRAO ou --cenario
Ordem parcial $\varphi_1 \prec_P \varphi_2$	lista ordem de cada cenário, com pares (φ_1, φ_2)
Pasta de saída por cenário	dicionário PASTA (situacao1/, situacao2/, situacao3/)

Tabela 5. Domínio e configuração: descrição formal $\rightarrow$ código.

Descrição formal	Código	Observação
$at(b, p, t)$	at[(b, p, t)]	nome no mapa: at(c, 0, 0)
$lev(b, l, t)$	lev[(b, l, t)]	lev(d, 1, 0)
$clr(b, t)$	clr[(b, t)]	clr(a, 3)
$mv(b, y, p, t)$	mv[(b, y, p, t)]	$t \in [0, H - 1]$; mv(d, c, 0, 0)
$cov(s, l, t)$	cov[(s, l, t)]	auxiliar: slot s coberto no nível l
(parte de cov)	cb[(b, s, l, t)]	auxiliar: o bloco b cobre s em l
$at \wedge lev$	pos(b, p, l, t)	auxiliar, só na ordem parcial da Situação 2
$o_t \leftrightarrow on(a, c, t)$	o_on(b, y, t)	auxiliar da ordem parcial
$on(b, y, t)$	não codificada	derivada em derivar_on() (interpretar.py)

Tabela 6. Variáveis: descrição formal $\rightarrow$ código.

Grupo	Bloco em <code>build()</code>	Comentário
—	definição de <code>cb</code> e <code>cov</code>	equivalências $cov(s, l, t) \leftrightarrow \bigvee_b cb(b, s, l, t)$ e $cb(b, s, l, t) \leftrightarrow lev(b, l, t) \wedge \bigvee_{p: s \in span(b, p)} at(b, p, t)$, usadas em 6 e 7
1	1. ESTADO INICIAL	unitárias de <code>at</code> e <code>lev</code> em $t = 0$
2	2. META	unitárias de <code>at</code> e <code>lev</code> em $t = H$
3, 4	3. UNICIDADE...	ao menos um e no máximo um ponto; idem para nível
5	5. EXCLUSAO HORIZONTAL	dois blocos no mesmo nível não cobrem o mesmo slot
6	6. ESTABILIDADE	uma cláusula por subconjunto de $\ell(b) - \lceil \ell(b)/2 \rceil + 1$ slots do span
7	7. CLEAR	$clr(b, t)$ definida nos dois sentidos, em todo instante
8	8. PRE-CONDICOES...	(a), (c), (d), (e) do texto; (b) removida (Seção 3.3); inclui o nível máximo e a cláusula do slot sob a ponte (marcada 3.4)
9	...EFEITOS	at, lev em $t + 1$ e $\neg clr(y, t + 1)$
10	10. FRAME AXIOMS	at e lev persistem se o bloco não se move
11	11. ACAO UNICA POR PASSO	exatamente um <code>mv</code> por instante
—	ORDEM PARCIAL	$\varphi_2(t) \rightarrow \varphi_1(0) \vee \dots \vee \varphi_1(t - 1)$

Tabela 7. Grupos de cláusulas: Seção 3.2 → código.

Cenário	H	Vars. base	Vars. auxiliares	Total de vars.	Cláusulas
Situação 1 ($S_0 \rightarrow S_{f4}$)	4	541	605	1146	31390
Situação 2 ($S_0 \rightarrow S_5$)	5	666	737	1403	38839
Situação 3 ($S_0 \rightarrow S_7$)	6	791	847	1638	46534

Tabela 8. Tamanho do CNF no menor horizonte satisfável. As variáveis base (at , lev , clr , mv) coincidem com a contagem das Seções 3.1 e 4.

3. Executar o miniSAT.

```
minisat situacao3/trab01_blocos2SAT.cnf situacao3/resultado3.txt
```

O miniSAT imprime estatísticas e a palavra SATISFIABLE (ou UNSATISFIABLE), e grava no arquivo as linhas SAT e a lista de inteiros. Ele relata menos cláusulas que o arquivo (32866 contra 46534) porque simplifica o CNF antes de resolver. O tempo de resolução é inferior a 0,1 s.

4. Interpretar o resultado.

```
python3 interpretar.py situacao3/resultado3.txt --verbose
```

PLANO ENCONTRADO (6 acoes):

1. t=0: mover bloco 'd' para CIMA de 'c' em p=0
2. t=1: mover bloco 'a' para CIMA de 'b' em p=5
3. t=2: mover bloco 'd' para a MESA em p=2
4. t=3: mover bloco 'a' para CIMA de 'c' em p=0
5. t=4: mover bloco 'b' para CIMA de 'c' em p=1
6. t=5: mover bloco 'd' para a MESA em p=3

ESTADO FINAL (t=6):

- a: ponto inicial p=0, nivel l=1
- b: ponto inicial p=1, nivel l=1
- c: ponto inicial p=0, nivel l=0
- d: ponto inicial p=3, nivel l=0

RELACOES 'on' em t=6:

- a esta sobre: c
- b esta sobre: c
- c esta na MESA
- d esta na MESA

Sem --map, o interpretar.py usa o trab01_blocos2SAT.map da mesma pasta do resultado. A opção --verbose também imprime a evolução do estado em cada instante.

6.1. Horizonte crescente

Para achar o plano mínimo, o horizonte é aumentado ($H = 0, 1, 2, \dots$) até o primeiro SATISFIABLE. O script rodar_cenarios.py faz esse laço: para cada H gera o CNF, chama o miniSAT, e ao encontrar o primeiro SAT salva .cnf, .map e resultadoN.txt na pasta do cenário.

```
python3 rodar_cenarios.py                # Situacoes 1 (Sf4), 2 e 3
python3 rodar_cenarios.py --todos        # inclui Sf1, Sf2 e Sf3
```

Os resultados são exatamente os previstos pelos planos manuais (Seção 4) e pela busca exaustiva sobre as mesmas regras (busca_exaustiva.py): 4, 5 e 6 ações para as três situações, e 9, 10 e 10 para S_{f1} , S_{f2} e S_{f3} .

6.2. Verificação independente

O `busca_exaustiva.py` implementa as pré-condições P1–P7 como um simulador de estados, sem SAT. Ele serve a dois fins: achar por BFS o plano mínimo de cada cenário e conferir, passo a passo, se um plano é legal e atinge a meta. O `rodar_cenarios.py` aplica essa verificação a todo plano devolvido pelo SAT, e todos passaram. Os planos manuais do relatório (Seções 4.1.2, 4.2.2 e 4.3.2) também são legais pelo mesmo simulador.

7. Interpretação da Saída do SAT Solver

7.1. O miniSAT não conhece nomes simbólicos

O miniSAT é um resolvidor puramente booleano. Para um CNF satisfatível, grava a palavra SAT e, na linha seguinte, uma lista de inteiros terminada em 0. Um inteiro positivo v significa que a variável v é verdadeira; um negativo $-v$, que é falsa. Por exemplo, o resultado da Situação 3 começa com:

```
SAT
-1 -2 -3 4 -5 -6 7 -8 -9 -10 -11 -12 -13 -14 -15 -16 17 18 ...
```

Aqui os valores verdadeiros 4, 7, 17 e 18 só ganham sentido com o `.map`: 4 at $(a, 3, 0)$ diz que a começa no ponto 3 em $t = 0$. Em toda a saída da Situação 3, 182 das 1638 variáveis são verdadeiras.

7.2. Da saída numérica ao plano em português

O `interpretar.py` executa três passos:

1. **Leitura do mapa** (`ler_mapa`): associa cada ID ao seu símbolo.
2. **Filtragem** (`ler_resultado`): mantém apenas os literais positivos. Se a primeira linha for UNSAT, avisa que não há plano e termina.
3. **Ordenação temporal** (`acoes, frase`): fica só com as variáveis $mv(b, y, p, t)$ verdadeiras, ordena por t e traduz cada uma em uma frase.

Na Situação 3, as seis variáveis mv verdadeiras são:

Conferências de sanidade. A contagem das variáveis verdadeiras por família confirma as restrições do CNF: há 28 *at* e 28 *lev* verdadeiras (4 blocos $\times$ 7 instantes, pela unicidade), exatamente 6 *mv* (uma por passo, pelo grupo 11), 19 *clr*, 49 *cov*, 49 *cb* e 3 *o_on* (auxiliares).

7.3. Derivação de *on*

A relação *on* não é codificada. O `interpretar.py` a reconstrói em `derivar_on` a partir de *at*, *lev* e da sobreposição de spans, como na Seção 2.2:

$$on(b, y, t) \leftrightarrow \exists p, q, l. at(b, p, t) \wedge at(y, q, t) \wedge lev(b, l, t) \wedge lev(y, l-1, t) \wedge overlap(b, p, y, q).$$

Na prática, para cada bloco b no estado de t :

- se $lev(b) = 0$, então $on(b, T, t)$ (sobre a mesa);
- se $lev(b) = l > 0$, procura todos os blocos y com $lev(y) = l - 1$ cujo span sobrepõe o de b . Pode haver mais de um: é a *ponte*.

Cenário	UNSAT em	SAT em	Variáveis	Cláusulas	Ações do plano
Situação 1 (S_{f4})	$H = 0, \dots, 3$	$H = 4$	1146	31390	4
Situação 2 (S_5)	$H = 0, \dots, 4$	$H = 5$	1403	38839	5
Situação 3 (S_7)	$H = 0, \dots, 5$	$H = 6$	1638	46534	6
Situação 1 (S_{f1})	$H = 0, \dots, 8$	$H = 9$	2366	68940	9
Situação 1 (S_{f2})	$H = 0, \dots, 9$	$H = 10$	2611	76481	10
Situação 1 (S_{f3})	$H = 0, \dots, 9$	$H = 10$	2611	76481	10

Tabela 9. Resultado do horizonte crescente. Em cada linha, vale o menor H satisfatível.

ID	Símbolo no .map	Frase
364	$mv(d, c, 0, 0)$	mover d para cima de c em $p = 0$
377	$mv(a, b, 5, 1)$	mover a para cima de b em $p = 5$
538	$mv(d, T, 2, 2)$	mover d para a mesa em $p = 2$
546	$mv(a, c, 0, 3)$	mover a para cima de c em $p = 0$
655	$mv(b, c, 1, 4)$	mover b para cima de c em $p = 1$
791	$mv(d, T, 3, 5)$	mover d para a mesa em $p = 3$

Tabela 10. Variáveis mv verdadeiras na Situação 3 e o plano que elas representam.

t	Estado (p, l)	Relações on derivadas
0	$a(3, 0), b(5, 0), c(0, 0), d(3, 1)$	d sobre a e b (ponte); a, b, c na mesa
1	$a(3, 0), b(5, 0), c(0, 0), d(0, 1)$	d sobre c ; a, b, c na mesa
6	$a(0, 1), b(1, 1), c(0, 0), d(3, 0)$	a e b sobre c ; c e d na mesa

Tabela 11. Relações on derivadas do estado reconstruído.

Como o estado de cada t é reconstruído de at e lev (estados_por_tempo), a mesma função vale para qualquer instante. Na Situação 3:

A opção `--verbose` imprime as relações *on* do estado final e a evolução (p, l) de cada bloco em todo t . A relação do estado inicial (a ponte de d sobre a e b) é obtida chamando `derivar_on` sobre o estado de $t = 0$.

7.4. Caso UNSATISFIABLE

Se o horizonte é menor que o comprimento mínimo do plano, o miniSAT responde UNSATISFIABLE e grava só UNSAT no arquivo. O `interpretar.py` avisa e termina com código de saída 1:

```
UNSATISFIABLE: nao existe plano com esse horizonte.
```

Isso é o resultado esperado para $H < \text{mínimo}$ (por exemplo, $H = 5$ na Situação 3) e é o que prova que o plano do menor H satisfatível é mínimo.

7.5. Regra de ouro

Sem o arquivo `.map`, a saída numérica do miniSAT é apenas uma lista de inteiros sem significado para o domínio. Além disso, o mapa precisa ser o da *mesma execução*: a numeração depende do cenário e do horizonte, pois as variáveis são criadas na ordem de t , bloco e posição. Por isso cada pasta `situacaoN/` guarda o par `.cnf/.map` junto ao `resultadoN.txt`.

Como demonstração, interpretar o resultado da Situação 3 com o mapa da Situação 2 não dá erro, mas devolve um plano sem sentido:

```
PLANO ENCONTRADO (16 acoes):
1. t=0: mover bloco 'a' para a MESA em p=0
2. t=0: mover bloco 'a' para a MESA em p=3
3. t=0: mover bloco 'a' para a MESA em p=4
...
```

São várias ações no mesmo instante $t = 0$, o que o grupo 11 proíbe. Esse tipo de saída indica mapa trocado.

7.6. Comparação com os planos manuais

Na Situação 2, o SAT escolheu um caminho diferente do manual. O manual (Seção 4.2.2) tira a de cima de c colocando-o na mesa em $p = 2$ e põe b sobre d ; o SAT põe a sobre d em $p = 3$ e leva b à mesa em $p = 2$. Daí em diante os planos coincidem: c sobe em d em $p = 4$, depois a e b sobem em c . Há, portanto, mais de um plano mínimo, e o SAT devolve um deles. Para os dois, a ordem parcial (primeiro c em $(4, 1)$, depois a e b sobre c) é respeitada.

Cenário	Ações (SAT)	Ações (manual)	Comparação
Situação 1 (S_{f4})	4	4	plano idêntico ao manual
Situação 2 (S_5)	5	5	planos diferentes, ambos legais e mínimos (abaixo)
Situação 3 (S_7)	6	6	plano idêntico ao manual

Tabela 12. Planos do SAT contra os planos manuais.